

ANIMAL SHELTER

SHAYKA ISLAM | 2007110

Oracle SQL Database Project | All SQL Operations | Animal Welfare

ANIMAL SHELTER, a non-profit organization, is created mainly for the stray animals of Bangladesh. we rescue, foster, give them home and give them up for adoption. we try to educate people about urgent veterinary help, vaccinations and sterilizations

PROJECT INSIGHT

RELATIONAL DATABASE

RELATIONS

1. Boxes
2. Animals
3. Adoptions
4. Owners
5. Employees
6. Duties
7. Donators
8. Donations

TOPICS

DDL

Create table, Add columns ,Modify Column Definition, Rename Column , Delete Column, Add Constraints,Create View,Drop Table,Drop view

DML

Data Insertion, Update record,Delete record

DQL

Select,Column Alias,Natural Join, Inner Join, Left Join ,Right Join, Full Join,Group By,Union,Union all,Intersect,Aggregate Function,Except,In,Not in,All,Any,Some, Where,Having, Like,Not Exists,Exis

PL/SQL

Exists,Exists,Nested query

OTHERS

Variable declare and manipulation,Loop, Conditional Logic, Exception Handling, Cursors, Stored Procedures and Functions , Trigger

Array with Extend,Array without Extend

HOW TO ASSIST

- Donate
- Adopt and take good care
- Volunteer
- Rescue Animals

ACTIVITIES

- Rescue homeless, attacked animals
- Taking Care of Animals
- Give Them Temporary Shelter
- Free Adoption to Animals
- Hire Volunteer



Animal Shelter is a database project using Oracle SQL. Where all the rescued homeless animals are listed ,along with the donations collected for them and who donated.Also volunteers along with their roles are also enlisted .People also can adopt those animals .So Adoption list ,and who adopted all are listed here.

Objectives:

- To design a relational database and apply all sql query,use pl/sql such that it can have real life application
- Data organization which promotes ease of access.
- Can develop any website or application on basis of this data source .
- Management of animals, their shelter, volunteers and their duty,adoption process,donation process.

Overview of Schema Boxes

- For each species there is a box and it has limited capacity constraints.
- **Columns:** 4 col, 1 pk
 - **box_id (Primary Key):** Unique identifier for each box.
 - **max_capacity:** Maximum capacity of the box.
 - **current_capacity:** Current number of animals in the box.
 - **species:** Type of species allowed in the box.
- **Constraints:**
 - Current_capacity cannot exceed the maximum_capacity or be less than zero.
 - Each box is assigned to a specific species.

Animals

- Stores information about individual animals.
- **Columns:** 12 col, 1 pk, 1 fk
 - **animal_id (Primary Key):** Unique identifier for each animal.
 - **animal_name:** Name of the animal.
 - **species:** Species of the animal.
 - **animal_status:** Health status of the animal among (Healthy, Sick, Pending).
 - **joined_at:** Date when the animal joined the shelter.



- **left_at:** Date when the animal is adopted
- **health:** Health condition of the animal (Good or Needs Attention).
- **box_id:** Foreign key linking the animal to a box.
- **breed, behaviour, descriptions:**
- **donation_status:** indicate ,if animal is donated or not or adopted

Constraints:

- animal status among Healthy, Sick, Pending
- joined_at date cant be null
- left at date can be null otherwise must be after join date
- Each animal is linked to a box and is subject to its capacity constraints.
- Donation status 1 means already donated once, 0 means not donated yet,-1 means animal is already adopted,cant be donated again

Trigger:

- **trigger_after_animal_deleted:** increase box capacity after an animal is deleted.
- **trg_before_animal_insert:** before animal insertion check box has capacity or not and if has capacity then insert and decrease.
- **trg_before_animal_delete:** adopted animals cant be deleted.

Adoptions

Tracks adoption records for each animal present in animals record.

Columns: 4 col, 1 pk, 1 fk

- **adoption_id (Primary Key):** Unique identifier for each adoption.
- **animal_id:** Foreign key linking the adoption to an animal (unique).
- **adoption_date:** Date of adoption.
- **descriptions:** Additional information about the

adoption.

Constraints:

- An animal can only be adopted once.

Trigger:

- **trigger_after_animal_adopted:** after insert on adoptions donation_status is set -1 of animals and current_capacity of boxes increases

Owners

Maintains information about who adopted the animals

Columns: 6 col, 1 pk, 1 fk

- **owner_id (Primary Key):** Unique identifier for each owner.
- **firstname, lastname:** Owner's name details.
- **owner_address:** Owner's address.
- **adoption_id:** Foreign key linking the owner to an adoption.
- **adopted_at:** Date when last adopted any animal.

Employees

Stores data about shelter employees.

Columns: 7 col, 1 pk

- **emp_id (Primary Key):** Unique identifier for each employee.
- **firstname, lastname:** Employee's name details.
- **salary:** Employee's salary.
- **joined_at:** Date when the employee joined.
- **left_at:** Date when the employee left.
- **roles:** Roles or responsibilities assigned to the employee.

Constraints:

- Employees must have valid salaries means >0
- Employment periods should follow logical constraints.



Duties

- Manages duties assigned to employees.

Columns:

- **duty_id (Primary Key):** Unique identifier for each duty.
- **emp_id:** Foreign key linking the duty to an employee.
- **box_id:** Foreign key linking the duty to a box.
- **weekdays:** How many days a week
- **start_at, end_at:** Start and end time of the duty.
- **responsibilities:** Description of the duty's responsibilities.

Constraints:

Duties must refer to an employee and a box.

Donators

- Stores details about individuals who donate to the shelter.

Columns:

- **donator_id (Primary Key):** Unique identifier for each donor.
- **firstname, lastname:** Donor's name details.
- **donator_address:** Address of the donor.
- **total_donations:** Sum of all donations made by donor.

Donations

- Tracks donations made by donators.

Columns:

- **donation_id (Primary Key):** Unique identifier for each donation.
- **donator_id:** Foreign key linking the donation to a donor.

- **animal_id:** Foreign key linking the donation to an animal.
- **amount:** Amount in taka donated.
- **date_of_donation:** Date when the donation was made.

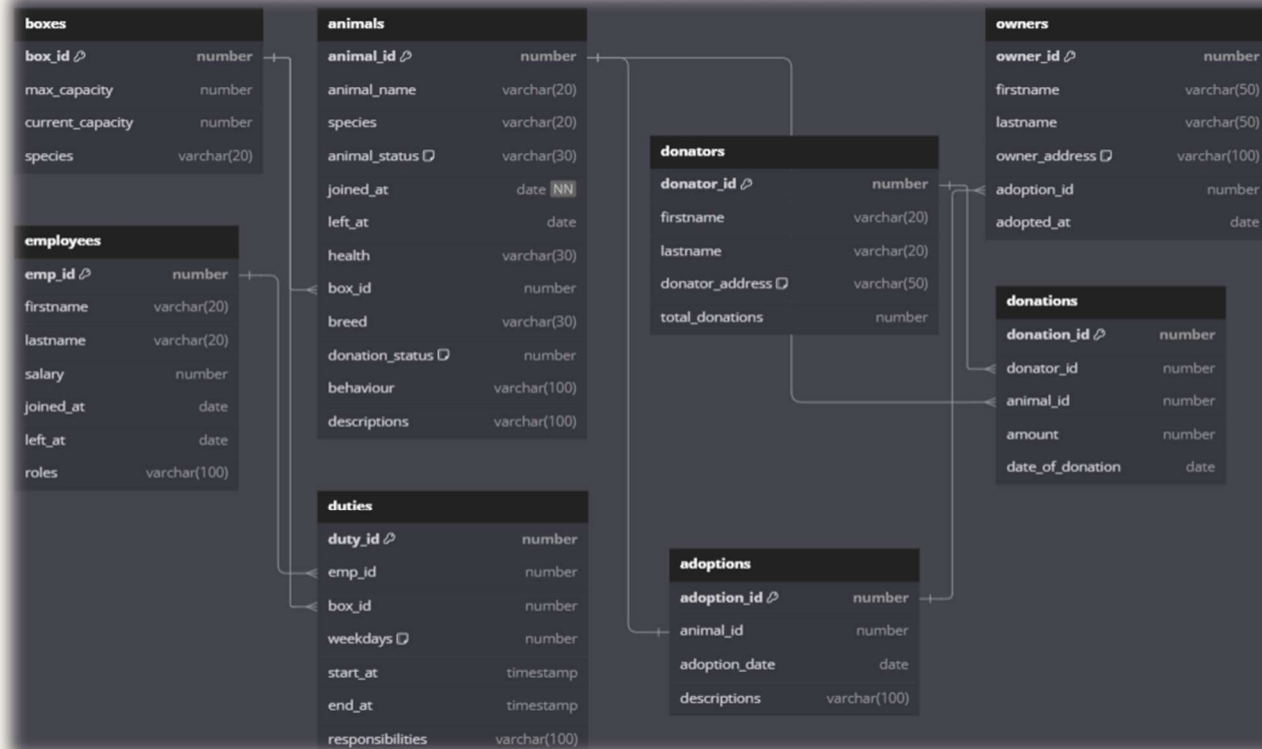
Constraints:

- One donation cant be made by multiple donators

Trigger:

- **trigger_before_donation_insert:** donation cant be done if an animal is adopted.
- **trg_after_donation_insert:** after donation donation status of animal must be changed to I.
- **trg_after_insert_donation:** after each donation amount donator total donation amount increased

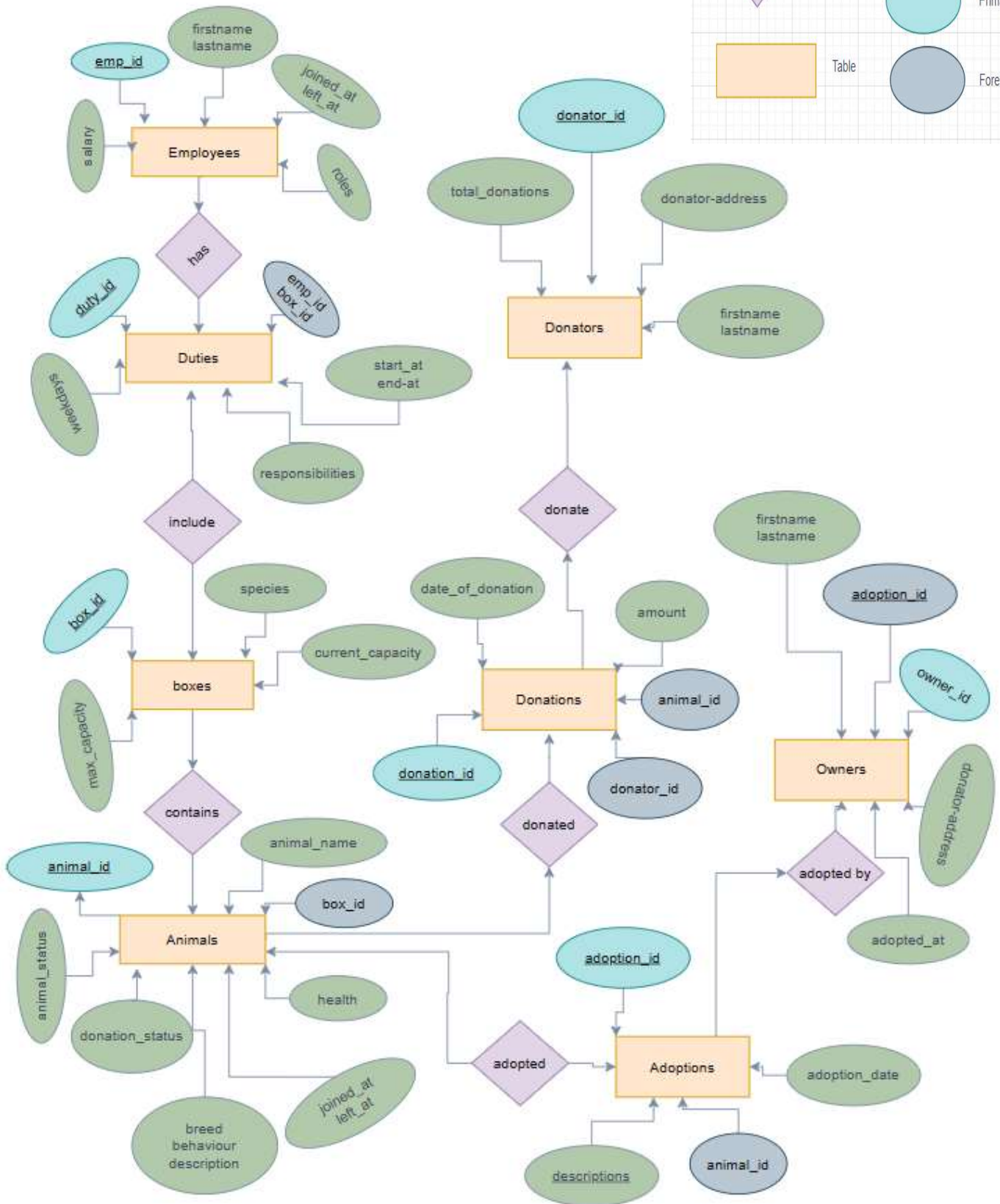
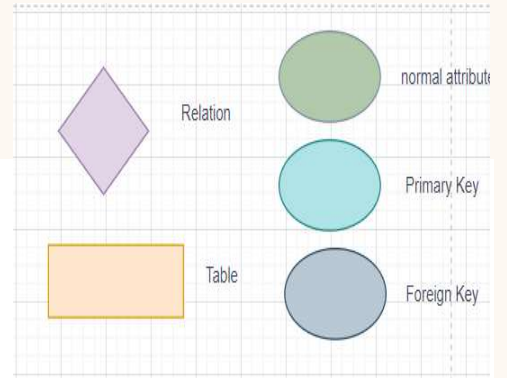
SCHEMA DIAGRAM



Cardinality

- Animals to Boxes:
 - Relationship: Many-to-One
 - Notation: Animals -> Boxes: Many same species animals can be kept in one box.
- Adoptions to Animals:
 - Relationship: One-to-One
 - Notation: Adoptions -> Animals: Each adoption record is for exactly one animal.
- Owners to Adoptions:
 - Relationship: One-to-Many
 - Notation: Owners -> Adoptions: Each owner has made multiple adoptions.
- Employees to Duties:
 - Relationship: One-to-Many
 - Notation: Employees -> Duties: One employee can perform many duties.
- Donators to Donations:
 - Relationship: One-to-Many
 - Notation: Donators -> Donations: One donator can make many donations.
- Donations to Animals:
 - Relationship: Many-to-One
 - Notation: Donations -> Animals: Many donations can support one specific animal.
- Duties to Boxes:
 - Relationship: Many-to-One
 - Notation: Duties -> Boxes: Many duties can be associated with one box (e.g., cleaning, feeding).

ER DIAGRAM



DDL (Data Definition Language)

```
CREATE TABLE boxes (  
    box_id NUMBER PRIMARY KEY,  
    max_capacity NUMBER CHECK (max_capacity>0),  
    current_capacity NUMBER,  
    species VARCHAR2(20),  
    CONSTRAINT capacity_check CHECK (current_capacity<=max_capacity AND current_capacity>=0)  
);  
CREATE TABLE animals (  
    animal_id NUMBER PRIMARY KEY,  
    animal_name VARCHAR2(20),  
    species VARCHAR2(20),  
    statuses VARCHAR2(30) CHECK (statuses IN ('Healthy','Sick','Pending') OR statuses IS NULL),  
    joined_at DATE NOT NULL,  
    left_at DATE,  
    health VARCHAR2(30) CHECK (health IN ('Good','Needs Attention')),  
    box_id NUMBER,  
    FOREIGN KEY (box_id) REFERENCES boxes(box_id) ON DELETE CASCADE,  
    CONSTRAINT check_animal_conditions CHECK (  
        (statuses='Healthy' AND health='Good') OR  
        (left_at IS NULL OR left_at>joined_at)  
    )  
);  
CREATE TABLE adoptions (  
    adoption_id NUMBER PRIMARY KEY,  
    animal_id NUMBER UNIQUE,  
    adoption_date date,  
    descriptions VARCHAR2(100),  
    FOREIGN KEY (animal_id) REFERENCES animals(animal_id) ON DELETE CASCADE  
);  
CREATE TABLE owners (  
    owner_id NUMBER primary key,  
    firstname VARCHAR2(50),  
    lastname VARCHAR2(50),  
    address VARCHAR2(100),  
    adoption_id NUMBER,  
    FOREIGN KEY (adoption_id) REFERENCES adoptions(adoption_id) ON DELETE CASCADE,  
    adopted_at date  
);  
CREATE TABLE employees (  
    emp_id NUMBER PRIMARY KEY,  
    firstname VARCHAR2(20),  
    lastname VARCHAR2(20),  
    salary NUMBER CHECK (salary>0),  
    joined_at DATE,  
    left_at DATE,  
    CONSTRAINT check_employee_dates CHECK (left_at IS NULL OR joined_at < left_at)  
);  
CREATE TABLE duties (  
    duty_id NUMBER PRIMARY KEY,  
    emp_id NUMBER,  
    FOREIGN KEY (emp_id) REFERENCES employees(emp_id) ON DELETE CASCADE,  
    box_id NUMBER,  
    FOREIGN KEY (box_id) REFERENCES boxes(box_id) ON DELETE CASCADE,
```

```

weekday NUMBER,
start_at NUMBER,
end_at NUMBER,
responsibilities VARCHAR2(100)
);

CREATE TABLE donators (
donator_id NUMBER PRIMARY KEY,
firstname VARCHAR2(20),
lastname VARCHAR2(20),
address VARCHAR2(50)
);

CREATE TABLE donations (
donation_id NUMBER PRIMARY KEY,
donator_id NUMBER,
FOREIGN KEY (donator_id) REFERENCES donators(donator_id) ON DELETE CASCADE,
animal_id NUMBER,
FOREIGN KEY (animal_id) REFERENCES animals(animal_id) ON DELETE CASCADE,
amount NUMBER CHECK (amount>0),
date_of_donation DATE
);
-- ADD COLUMN
ALTER TABLE animals ADD breed VARCHAR2(30) NOT NULL;
ALTER TABLE animals ADD donation_status NUMBER DEFAULT 0 CHECK (donation_status IN (-1,0,1));
ALTER TABLE animals ADD behaviour VARCHAR2(100);
ALTER TABLE animals ADD descriptions VARCHAR2(100);
ALTER TABLE animals ADD picture BLOB;
ALTER TABLE employees ADD roles VARCHAR2(100);
ALTER TABLE donators ADD total_donations NUMBER CHECK (total_donations>=0);
--MODIFY COLUMN Definition
ALTER TABLE duties MODIFY start_at TIMESTAMP;
ALTER TABLE duties MODIFY end_at TIMESTAMP;
--RENAME the COLUMN
ALTER TABLE animals RENAME COLUMN statuses TO animal_status;
ALTER TABLE duties RENAME COLUMN weekday TO weekdays;
ALTER TABLE owners RENAME COLUMN address TO owner_address;
ALTER TABLE donators RENAME COLUMN address TO donator_address;
--delete COLUMN
ALTER TABLE animals DROP COLUMN picture;
--ADD constraint
ALTER TABLE duties ADD CONSTRAINT check_dates CHECK (end_at IS NULL OR end_at>start_at);

```

DML (Data Modification Language)

```

--Update operation
UPDATE employees SET roles='General Volunteer' WHERE emp_id=2;
UPDATE owners SET owner_address='Fulbarigate Street' WHERE owner_id=3;
UPDATE donations SET amount=2000 WHERE donation_id=5;
UPDATE boxes SET max_capacity=140 WHERE box_id=1;
UPDATE animals SET box_id=(SELECT box_id FROM boxes WHERE species='Fishes') WHERE box_id=6;

--delete operation
--delete empty box from boxes
DELETE FROM boxes WHERE box_id NOT IN (SELECT DISTINCT box_id FROM animals);
--delete donators with no donation
DELETE FROM donators WHERE donator_id NOT IN (SELECT DISTINCT donator_id FROM donations);

```


Triggers

```
-- Trigger to increase box capacity after an animal is deleted
CREATE OR REPLACE TRIGGER trigger_after_animal_deleted
AFTER DELETE ON animals
REFERENCING OLD AS o
FOR EACH ROW
BEGIN
    UPDATE boxes SET current_capacity=current_capacity+1
    WHERE box_id=:o.box_id;
END;
/

--trigger after adoptions donaation status box capacity
--increase box capacity
CREATE OR REPLACE TRIGGER trigger_after_animal_adopted
AFTER INSERT ON adoptions
REFERENCING OLD as o NEW AS n
FOR EACH ROW
BEGIN
    UPDATE animals SET donation_status=-1 WHERE animal_id=:n.animal_id;
    UPDATE boxes b SET b.current_capacity=b.current_capacity+1
    WHERE b.box_id=(SELECT a.box_id FROM animals a WHERE a.animal_id=:n.animal_id);
END;
/

--before animal insertion check box has capacity or not
--and if has capacity then insert and decrease
CREATE OR REPLACE TRIGGER trg_before_animal_insert
BEFORE INSERT ON animals
REFERENCING OLD as o NEW AS n
FOR EACH ROW
DECLARE
    current_capacity NUMBER;
    box_species VARCHAR2(20);
BEGIN
    SELECT current_capacity,species INTO current_capacity,box_species
    FROM boxes
    WHERE box_id=:n.box_id;
    IF current_capacity=0 THEN
        RAISE_APPLICATION_ERROR(-20001,'the box is full,create a box record 1st');
    ELSE
        UPDATE boxes b SET b.current_capacity=b.current_capacity-1 WHERE b.box_id=:n.box_id;
    END IF;
    IF NOT(box_species=:n.species OR box_species='Others') THEN
        RAISE_APPLICATION_ERROR(-20002,'Place in Correct Box');
    END IF;
END;
/

--donation cant be done if an animal is adopted
CREATE OR REPLACE TRIGGER trigger_before_donation_insert
BEFORE INSERT ON donations
REFERENCING OLD as o NEW AS n
FOR EACH ROW
DECLARE
    don_status NUMBER;
BEGIN
    IF :n.animal_id IS NOT NULL THEN
        SELECT donation_status INTO don_status FROM animals WHERE animal_id=:n.animal_id;
```

```

animal_id=:n.animal_id;
    IF don_status=-1 THEN
        RAISE_APPLICATION_ERROR(-20003,'This animal has already been adopted and cannot receive
further donations');
    END IF;
END IF;
END;
/
--adopted animals cant be deleted
CREATE OR REPLACE TRIGGER trg_before_animal_delete
BEFORE DELETE ON animals
REFERENCING OLD AS o NEW AS n
FOR EACH ROW
BEGIN

    IF :o.donation_status=-1 THEN
        RAISE_APPLICATION_ERROR(-20004,'Cannot delete an adopted animal');
    END IF;
END;
/
--after each donation amount donator total donation amount increased
CREATE OR REPLACE TRIGGER trg_after_insert_donation
AFTER INSERT ON donations
REFERENCING OLD AS o NEW AS n
FOR EACH ROW
BEGIN
    UPDATE donators SET total_donations=total_donations+:n.amount
    WHERE donator_id=:n.donator_id;
END;
/
--after donation donation status of animal must be changed
CREATE OR REPLACE TRIGGER trg_after_donation_insert
AFTER INSERT ON donations
REFERENCING OLD AS o NEW AS n
FOR EACH ROW
BEGIN
    IF :n.animal_id IS NOT NULL THEN
        UPDATE animals
        SET donation_status=1
        WHERE animal_id=:n.animal_id;
    END IF;
END;
/

```

SQL Queries

```
SELECT * FROM boxes;
SELECT * FROM animals WHERE box_id=(SELECT box_id FROM boxes WHERE species='Mammals');
SELECT * FROM animals;
SELECT * FROM adoptions;
SELECT * FROM owners;
SELECT * FROM employees;
--Use of column alias
SELECT emp_id AS EMPLOYEE_ID,lastname AS "LAST NAME",(salary - 1000) * 5 AS Modified_Salary
FROM employees;
SELECT emp_id AS EMPLOYEE_ID,lastname AS LAST_NAME,salary * 12 AS ANNUAL_SALARY FROM employees;
SELECT * FROM duties;
SELECT * FROM donators;
--Use of column alias
SELECT donator_id,firstname||' '||lastname AS FULL_NAME,total_donations AS ANNUAL_DONATION FROM
donators;
SELECT * FROM donations;
--Union
--List all unique names both of animals and of donators
--union always returns distinct rows.
--if we didnot use AS ,then col name would be the 1st one ,,animal_name
SELECT animal_name AS Name FROM animals UNION SELECT firstname AS Name FROM donators;
--Union All
--Includes duplicate values
SELECT animal_name FROM animals WHERE species = 'Cat' UNION ALL SELECT animal_name FROM animals
WHERE species = 'Dog';
--Intersection
--names that are both an animal's name and a donator's first name
SELECT animal_name FROM animals INTERSECT SELECT firstname FROM donators;
--Except
--names those are not donated
--MINUS is equivalent to EXCEPT
SELECT animal_name FROM animals MINUS SELECT animal_name FROM animals WHERE animal_id
IN(SELECT animal_id FROM donations);
--With clause
--List of Animals and Their Donation Totals
WITH DonationTotal AS(SELECT animal_id,SUM(amount) AS totalDonation FROM donations GROUP BY
animal_id)SELECT * FROM DonationTotal;
```

Aggregate Functions

```
--Aggregate function
--Count how many animals are healthy.
SELECT COUNT(*) AS "Healthy Animal" FROM animals WHERE health='Healthy';
--Count the number of donations greater than $500.
SELECT COUNT(*) AS "Donation > 500" FROM donations WHERE amount>500;
--Count how many distinct species are present in the shelter.
SELECT COUNT(DISTINCT species) AS "distinct species count" FROM animals;
--Count the number of animals that have received at least one donation.
SELECT COUNT(DISTINCT animal_id) AS animals_with_donations FROM donations WHERE animal_id IS
NOT NULL;
--Count the number of animals in each box,highlighting boxes with more than 5 animals.
SELECT box_id,COUNT(*) AS "animal count >5" FROM animals GROUP BY box_id HAVING COUNT(*) > 5;
--Calculate the total sum of donations
SELECT SUM(amount) AS total_donations FROM donations;
--Calculate total sum of donation for each animal
SELECT animal_id,SUM(amount) AS total_donations FROM donations GROUP BY animal_id;
--Determine the maximum capacity among all boxes.
SELECT MAX(max_capacity) AS "max box capacity" FROM boxes;
```

```
--Calculate the average current capacity utilization of boxes.
SELECT AVG(current_capacity) AS avg_animals_per_box FROM boxes;
SELECT
    SUM(amount) AS total_donations,
    MIN(amount) AS smallest_donation,
    MAX(amount) AS largest_donation,
    COUNT(*) AS total_number_of_donations
FROM donations;
```

WITH, Group By, Having

```
--Calculate the average amount of the largest donation received for each animal
WITH max_donation AS(
    SELECT animal_id, MAX(amount) AS largest_donation
    FROM donations
    GROUP BY animal_id
)
SELECT AVG(largest_donation) AS average_largest_donation FROM max_donation;
--Group By and Having
--Identify species with more than a certain number of animals
--based on a specified donation
SELECT species, COUNT(*) AS num_animals FROM animals WHERE donation_status > 0 GROUP BY species
HAVING COUNT(*) > 1;

--Set Membership(And, or, NOT)
-- Find animals that are 'Healthy' and have received donations
SELECT animal_name, health FROM animals WHERE health = 'Healthy' AND animal_id IN(SELECT animal_id FROM donations);
--Find animals that are either 'Mammals' or 'Birds'
SELECT animal_name, species FROM animals WHERE species = 'Mammals' OR species = 'Birds';
--Find all donations not made by a specific donator
SELECT donation_id, amount FROM donations WHERE donator_id NOT IN(SELECT donator_id FROM donators WHERE lastname = 'Khan');
--Find animals that are 'Healthy' and have received donations but are not 'Cats'
SELECT animal_name FROM animals WHERE health = 'Healthy' AND animal_id IN(SELECT animal_id FROM donations) AND breed != 'Cat';
SELECT animal_name FROM animals WHERE health = 'Healthy' AND animal_id IN(SELECT animal_id FROM donations) AND breed NOT IN('Cat');
SELECT animal_name FROM animals WHERE breed NOT LIKE 'Cat';
--Some
--Find all donations that are greater than the avg donation made for any animal
SELECT * FROM donations where amount > SOME(SELECT AVG(amount) FROM donations GROUP BY animal_id);
--All
--Find the employees where salary is greater than all the min salary for each role
SELECT * FROM employees WHERE salary > ALL(SELECT MIN(salary) FROM employees GROUP BY roles);
--Not Exists
--Find all employees who have never been assigned any duties
SELECT * FROM employees WHERE NOT EXISTS (SELECT * FROM duties WHERE emp_id = employees.emp_id);
--Exists
--Find all employees who have been assigned any duties
SELECT * FROM employees WHERE EXISTS (SELECT * FROM duties WHERE emp_id = employees.emp_id);
--Unique
--Find the count of animals that have received donations
SELECT COUNT(DISTINCT animal_id) AS donated_animals FROM donations;
```


String Operations

```
--Retrieve animals whose names start with 'B', followed by any 3 char, and then 'l'
SELECT * FROM animals WHERE animal_name LIKE 'B___l%';
--Find animals whose names start with "S"
SELECT * FROM animals WHERE animal_name LIKE 'S%';
--List animals whose names contain the substring "ir"
SELECT * FROM animals WHERE animal_name LIKE '%ir%';
--Find animals that joined in January 2023
SELECT * FROM animals WHERE joined_at BETWEEN TO_DATE('2023-01-01', 'YYYY-MM-DD') AND
TO_DATE('2023-01-31', 'YYYY-MM-DD');
--Search for employees whose last name has "Khan" substr
SELECT * FROM employees WHERE lastname LIKE '%Khan%';
--Find donators whose firstname ends with m and has 5 letters
SELECT * FROM donators WHERE firstname LIKE '____m';
--Select donators whose first name starts with 'M', followed by any character, and then 'n'
SELECT * FROM donators WHERE firstname LIKE 'M_n%';
```

Join, Nested Query

```
--Join operations
--Natural Join
--based on common column of both tables
SELECT * FROM employees NATURAL JOIN duties;
--Inner Join or JOIN
SELECT a.animal_name, a.species, ad.descriptions AS adoption_description, o.firstname || '
' || o.lastname AS owner_name
FROM animals a
INNER JOIN adoptions ad ON a.animal_id=ad.animal_id
INNER JOIN owners o ON ad.adoption_id=o.adoption_id;
--Left Join
--Names, Species, and Adoption descriptions of all animals in the database, including those
that haven't been adopted yet
SELECT a.animal_name, a.species, ad.descriptions AS adoption_description
FROM animals a
LEFT JOIN adoptions ad ON a.animal_id=ad.animal_id;
--Right Join
--all boxes and their contained animals. If some boxes have no animals assigned to them
SELECT b.box_id, b.species AS box_species, a.animal_name
FROM boxes b
RIGHT JOIN animals a ON b.box_id=a.box_id;
--Full Join
--all animals and their adoption status, including animals that haven't been adopted and
adoption records without a current animal match
SELECT a.animal_name, ad.adoption_date, ad.descriptions
FROM animals a
FULL JOIN adoptions ad ON a.animal_id=ad.animal_id;

--the total number of employees who have a salary greater than the average salary of employees
in the 'Adoption Coordinator' role
SELECT COUNT(*) AS FROM employees WHERE salary > (SELECT AVG(salary) FROM employees WHERE
roles='Adoption Coordinator'
);
```

Procedures & Functions

```
--Delete record Procedure
-- Procedure to delete from boxes
CREATE OR REPLACE PROCEDURE delete_box(id IN boxes.box_id%TYPE)
AS
BEGIN
    DELETE FROM boxes WHERE box_id=id;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Delete failed from boxes');
END;
/

--Functions total present animals in a specific box
--adopted animals are not included
CREATE OR REPLACE FUNCTION animal_cnt_box(id boxes.box_id%TYPE)
RETURN NUMBER AS
    max_cap NUMBER;
    current NUMBER;
    net NUMBER;
BEGIN
    SELECT current_capacity INTO current FROM boxes WHERE box_id=id;
    SELECT max_capacity INTO max_cap FROM boxes WHERE box_id=id;
    net:=max_cap-current;
    RETURN net;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END;
/
```

Conclusion:

Throughout the project, we successfully developed a normalized database schema that reduced redundancy and increased data integrity. We implemented SQL triggers to maintain consistency across the database and to automate tasks that are critical for the shelter's operation. These enhancements have not only increased the efficiency of the database but have also simplified the user interaction with the system, making it easier for the shelter staff to track, update, and retrieve information swiftly and accurately.

The End